

Solutions to Week 12 conceptual questions

Objectives: To develop an understanding of bipartite graph matching and how the Hungarian Algorithm solves the minimum-cost bipartite matching problem and its link to linear programming and network flow problems.

Conceptual exercises

1. Given an $N \times N$ cost matrix $C = [c_{ij}]$ and its associated bipartite graph/clique, formulate the minimum-cost bipartite matching problem using binary decision variables $X = [x_{ij}]$ as a mathematical optimization problem.

Ans: 1

Let Ω denote the set of all perfect matchings in the given bipartite graph. Each perfect matching $M \in \Omega$ can be represented either as a list of N assigned pairs of vertices $[(i, j)]$ or equivalently by an $N \times N$ binary decision matrix $X(M) = [x_{ij}]$, where each $x_{ij} = \{0, 1\}$.

The cost of any perfect matching M is given by

$$\text{cost}(M) = \sum_{\forall (i,j) \in M} c_{ij} = \sum_{i=1}^N \sum_{j=1}^N c_{ij} x_{ij}.$$

The matching must satisfy the following constraints:

$$\sum_{j=1}^N x_{ij} = 1 \quad \forall 1 \leq i \leq N.$$

(In words, this is saying each vertex i in the first partition of the bipartite graph must be matched with exactly one vertex j in the second partition.)

$$\sum_{i=1}^N x_{ij} = 1 \quad \forall 1 \leq j \leq N.$$

(In words, this is saying each vertex j in the second partition must be matched with exactly one vertex i in the first partition.)

The goal/objective for this problem is to find a perfect matching M^* such that

$$M^* = \arg \min_{M \in \Omega} \text{cost}(M).$$

(That is, M^* is a perfect matching that has the minimum cost over all possible matchings in the set Ω .)

2. Let $\{u_i\}$ and $\{v_j\}$ be dual variables associated with the rows and columns of $C = [c_{ij}]$.

- (a) Show that the objective function of the linear assignment problem can be transformed into the following form:

$$z = \sum_{i=1}^N \sum_{j=1}^N (c_{ij} - u_i - v_j) x_{ij} + \sum_{i=1}^N u_i + \sum_{j=1}^N v_j.$$

Ans: 2(a)

The cost function in this problem is:

$$z = \sum_{i=1}^N \sum_{j=1}^N x_{ij} c_{ij}$$

For a given bipartite graph matching problem of finite size N , introduce the row and column dual variables as follows, and perform the following algebraic transformations:

$$\begin{aligned} z &= \sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j + u_i + v_j) \\ &= \sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j) + \sum_{i=1}^N \sum_{j=1}^N x_{ij} u_i + \sum_{i=1}^N \sum_{j=1}^N x_{ij} v_j \\ &= \sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j) + \sum_{i=1}^N \sum_{j=1}^N x_{ij} u_i + \sum_{\textcolor{red}{j}=1}^N \sum_{i=1}^N x_{ij} v_j \\ &= \sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j) + \sum_{i=1}^N u_i \underbrace{\left(\sum_{j=1}^N x_{ij} \right)}_{=1} + \sum_{j=1}^N v_j \underbrace{\left(\sum_{i=1}^N x_{ij} \right)}_{=1} \\ &= \sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j) + \sum_{i=1}^N u_i + \sum_{j=1}^N v_j \end{aligned}$$

- (b) Assume that $c_{ij} = u_i + v_j$ whenever $x_{ij} = 1$. Reason why under this equality condition the objective simplifies to

$$z = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j.$$

Ans: 2(b)

We showed above that

$$z = \underbrace{\sum_{i=1}^N \sum_{j=1}^N x_{ij} (c_{ij} - u_i - v_j)}_{\text{Term 1}} + \sum_{i=1}^N u_i + \sum_{j=1}^N v_j.$$

The first term on the right-hand side sums $x_{ij}(c_{ij} - u_i - v_j)$ over all values of i, j .

For all pairs (i, j) where $x_{ij} = 0$, the contribution to Term 1 is 0.

For all pairs (i, j) where $x_{ij} = 1$, we are given that $c_{ij} = u_i + v_j \implies c_{ij} - u_i - v_j = 0$, so the contribution to Term 1 is again 0.

Since x_{ij} are binary decision variables in this problem the objective simplifies to $z = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$.

- (c) Assume that $c_{ij} \geq u_i + v_j$ whenever $x_{ij} = 0$ and $c_{ij} = u_i + v_j$ whenever $x_{ij} = 1$. Show that the minimum-cost bipartite matching problem can be equivalently modelled as

$$\max \quad \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$$

subject to

$$u_i + v_j \leq c_{ij} \quad \forall i, j.$$

Ans: 2(c)

Let Ω denote the set of all perfect matchings in the given bipartite graph. For any perfect matching $M \in \Omega$ we have

$$\text{Cost}(M) = \sum_{\forall (i,j) \in M} c_{ij}.$$

Since $c_{ij} \geq u_i + v_j$, we have

$$\text{Cost}(M) = \sum_{\forall (i,j) \in M} c_{ij} \geq \sum_{\forall (i,j) \in M} (u_i + v_j).$$

Since M is a perfect matching, we have

$$\sum_{\forall (i,j) \in M} (u_i + v_j) = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j.$$

and therefore

$$\text{Cost}(M) \geq \sum_{i=1}^N u_i + \sum_{j=1}^N v_j. \quad (1)$$

But, suppose there exists a perfect matching M^* such that for every edge $(i, j) \in M^*$ we observe a strict equality $c_{ij} = u_i + v_j$, then

$$\text{Cost}(M^*) = \sum_{\forall (i,j) \in M^*} c_{ij} = \sum_{\forall (i,j) \in M^*} (u_i + v_j) = \sum_{i=1}^N u_i + \sum_{j=1}^N v_j.$$

Then from the bound defined by Eq 1, it must be that $\text{Cost}(M^*)$ on the lhs achieves the minimum value over all possible perfect matchings $M \in \Omega$, and simultaneously $\sum_{i=1}^N u_i + \sum_{j=1}^N v_j$ on the rhs achieves the maximum value.

Therefore, the minimum-cost bipartite matching problem (modelled in Q1) is equivalent to solving $\max \sum_{i=1}^N u_i + \sum_{j=1}^N v_j$ subject to $u_i + v_j \leq c_{ij}, \forall i, j$.

3. A company has four trucks located at different garages. Each truck must be assigned to exactly one of four different sites, with no site receiving more than one truck. The distances (in kilometers) between the truck locations and the sites are given below.

	Site A	Site B	Site C	Site D
Truck 1	90	75	75	80
Truck 2	35	85	55	65
Truck 3	125	95	90	105
Truck 4	45	110	95	115

Assign each truck to a site so as to minimize the total distance traveled.

Ans: 3

Original cost matrix:

	0	0	0	0
0	90	75	75	80
0	35	85	55	65
0	125	95	90	105
0	45	110	95	115

(Note: $\sum u_i + \sum v_j = 0$. $|M| = 0$)

After Step 1:

	0	0	0	0
75	15	0	0	5
35	0	50	20	30
90	35	5	0	15
45	0	65	50	70

After Step 2:

	0	0	0	5
75	15	0	0	0
35	0	50	20	25
90	35	5	0	10
45	0	65	50	65

(Note: $\sum u_i + \sum v_j = 250$. $|M| = 0$)

After Step 3.1:

	0	0	0	5
75	15	0	$\emptyset$	0
35	0	50	20	25
90	35	5	0	10
45	$\emptyset$	65	50	65

After Step 3.2:

	0	0	0	5
75	15	0	$\emptyset$	$\emptyset$
35	0	50	20	25
90	35	5	0	10
45	$\emptyset$	65	50	65

Before going to step 3.3, ensure all 0-cells have either been assigned or crossed out. (Yes, in the above state, continue to step 3.3.)

After Step 3.3:

	0	0	0	5	
75	15	0	$\emptyset$	$\emptyset$	
35	0	50	20	25	✓
90	35	5	0	10	
45	$\emptyset$	65	50	65	✓
	✓				

$\sum u_i + \sum v_j = 250$.

$|M| = 3$ (perfect matching not yet reached)

After Step 3.4:

	0	0	0	5	
75	15	0	$\emptyset$	$\emptyset$	
35	0	50	$\underbrace{20}_{\theta}$	25	✓
90	35	5	0	10	
45	$\emptyset$	65	50	65	✓
	✓				

After Step 4 (= start of **iteration 2**)

	-20	0	0	5
75	35	0	0	0
55	0	30	0	5
90	55	5	0	10
65	0	45	30	45

After Step 3.1 (**iteration 2**)

	-20	0	0	5
75	35	0	$\emptyset$	0
55	$\emptyset$	30	$\emptyset$	5
90	55	5	0	10
65	0	45	30	45

After Step 3.2 (**iteration 2**)

	-20	0	0	5
75	35	0	$\emptyset$	$\emptyset$
55	$\emptyset$	30	$\emptyset$	5
90	55	5	0	10
65	0	45	30	45

Before going to step 3.3, ensure all 0-cells have either been assigned or crossed out. (Yes, in the above state, continue to step 3.3.)

After Step 3.3 (**iteration 2**)

	-20	0	0	5	
75	35	0	$\emptyset$	$\emptyset$	
55	$\emptyset$	30	$\emptyset$	5	✓
90	55	5	0	10	✓
65	0	45	30	45	✓
	✓		✓		

$\sum u_i + \sum v_j = 270$.
 $|M| = 3$ (perfect matching not yet reached)

After Step 3.4 (**iteration 2**)

	-20	0	0	5	
75	35	0	$\emptyset$	$\emptyset$	
55	$\emptyset$	30	$\emptyset$	$\underbrace{5}_{\theta}$	✓
90	55	5	0	10	✓
65	0	45	30	45	✓
	✓		✓		

After step 4 (**iteration 2**) = Start of **iteration 3**

	-25	0	-5	5
75	40	0	5	0
60	0	25	0	0
95	55	0	0	5
70	0	40	30	40

After steps 3.1-3.2 (**iteration 3**)

	-25	0	-5	5
75	40	0	5	0
60	$\emptyset$	25	0	0
95	55	0	0	5
70	0	40	30	40

At this stage, not all 0-cells have been assigned or crossed out, and the assignment is incomplete. This indicates multiple optimal solutions exist for the problem. To proceed, arbitrarily select one of the remaining unassigned zero cells and continue the assignment process.

Say you arbitrarily assigned (1,2). Once you assign (1,2), strike out all 0-cells in the corresponding row AND column.

	-25	0	-5	5
75	40	0	5	$\emptyset$
60	$\emptyset$	25	0	0
95	55	$\emptyset$	0	5
70	0	40	30	40

Repeat steps 3.1-3.2 (**iteration 3**) to explore newer assignments.

	-25	0	-5	5
75	40	0	5	$\emptyset$
60	$\emptyset$	25	$\emptyset$	0
95	55	$\emptyset$	0	5
70	0	40	30	40

$$\sum u_i + \sum v_j = 275.$$

$|M| = 4$. Perfect matching! **Stop!** Optimality reached. Optimal min-cost bipartite graph matching: $\{(1, 2), (2, 4), (3, 3), (4, 1)\}$

(Sanity check that the sum of costs in the original cost matrix (at the start) of these cells adds up to the sum of dual variables shown above.)

- A job consultant has four clients (each advertising a job) and five applicants looking to be hired. The consultant is able to reliably rank the suitability of each applicant to the clients' job description on a scale of zero (poorest fit) to ten (best fit). The consultant's rankings are given in the following table.

	Appl. 1	Appl. 2	Appl. 3	Appl. 4	Appl. 5
Client 1	7	4	7	3	10
Client 2	5	9	3	8	7
Client 3	3	5	6	2	9
Client 4	6	5	0	4	8

The consultant wants to match the applicants to clients and in doing so aims to **maximize** (not minimize) the sum of the matched rankings. Can this be solved using the Hungarian approach? If so, how is the above problem converted to an instance of min-cost bipartite graph matching to be able to do so?

Ans: 4

(answered in a plain text format)

Job ranking (weights):

7	4	7	3	10
5	9	3	8	7
3	5	6	2	9
6	5	0	4	8

The problem requires maximizing weights.

Multiply by -1 (convert max weight to min cost problem)

-7	-4	-7	-3	-10
-5	-9	-3	-8	-7
-3	-5	-6	-2	-9
-6	-5	0	-4	-8

Easier to work with costs ≥ 0 ;

Subtract by min value in the current matrix (which is -10).

Implies add 10 to all (i,j):

3	6	3	7	0
5	1	7	2	3
7	5	4	8	1
4	5	10	6	2

To use Hungarian, need a square matrix.

Add dummy row with 0s:

3	6	3	7	0
5	1	7	2	3
7	5	4	8	1
4	5	10	6	2
0	0	0	0	0

Step 1 for feasible u_i 's (reduce by row min)

0	3	6	3	7	0
1	4	0	6	1	2
1	6	4	3	7	0
2	2	3	8	4	0
0	0	0	0	0	0

Step 2 for feasible v_j s

	0	0	0	0	0

0	3	6	3	7	0
1	4	0	6	1	2
1	6	4	3	7	0
2	2	3	8	4	0
0	0	0	0	0	0

Step 3 (x = crossed out 0s)

	0	0	0	0	0	marking

0	3	6	3	7	[0] !	
1	4	[0]	6	1	2	--->unmarked row
1	6	4	3	7	x	!
2	2	3	8	4	x	!
0	[0]	x	x	x	x	--->unmarked row

!

Step 4 Theta = 2

	0	0	0	0	-2

2	1	4	1	5	0
1	4	0	6	1	4
3	4	2	1	5	0
4	0	1	6	2	0
0	0	0	0	0	2

```

Repeat step 3:
  0   0   0   0  -2
-----
2|  1   4   1   5  [0]!!
1|  4  [0]   6   1   4  |--->unmarked
3|  4   2   1   5   x  |!
4|  [0]   1   6   2   x  |--->unmarked
0|  x   x  [0]   x   2  |--->unmarked
-----

```

```

!
Step 4 theta = 1:
  0   0   0   0  -3
-----
3|  x   3   0   4   0
1|  4  [0]   6   1   5
4|  3   1   0   4   0
4|  [0]  1   6   2   1
0|  x   x   x  [0]   3
-----

```

Not all 0s crossed (x) out or assigned.
 Break the impasse by choosing an arbitrary assignment.
 choose (1,3) cross out other 0s in row 1 and column 3.
 Repeat steps 3.1 and 3.2

```

  0   0   0   0  -3
-----
3|  x   3  [0]   4   x
1|  4  [0]   6   1   5
4|  3   1   x   4  [0]
4|  [0]  1   6   2   1
0|  x   x   x  [0]   3
-----

```

|M| = 5. STOP.

Assignment
 (1,3), (2,2), (3,5), (4,1), (5,4)--> dummy
 Max weight
 = 7 + 9 + 9 + 6 + 0 = 31

Sanity check:
 $\sum u_i + \sum v_j = 9$.
 This should be the same as values in the cells where the assignment has been made using transformed cost matrix (not original weight matrix): $3+1+1+4 = 9$.

Further, during transformation from weights to cost, we also added +10 to all values in the matrix after multiplying weights by -1. Perfect matching has 4 actual assignments which each got added 10 (plus 1 dummy one added later, whose cost was set to 0, so no effect of it). Therefore, inverting the transformation to see the equivalence: $\text{Max weight} = (9-4*10)*-1 = -(9-40) = 31$

The following questions are intended to highlight connections with the Network Flows topic covered in FIT2004. FIT3155 will not assess knowledge of network flows and its applications in the final exam, as this is assumed prior knowledge. However, as part of your learning it remains useful to understand these connections.

- The above questions were instances of minimum-cost bipartite matching problems. A related variant is the maximum **cardinality** bipartite matching problem, whose objective is to match as many vertices as possible in a bipartite graph such that each vertex is incident to at most one matched edge in the resultant matching. How can a network flow formulation be adapted to solve this problem?

Ans: 5

A maximum-cardinality bipartite matching problem can be converted into a network flow problem as follows.

Consider a bipartite graph $G(L \cup R, E)$, where $L = \{\ell_1, \ell_2, \dots, \ell_m\}$ and $R = \{r_1, r_2, \dots, r_n\}$.

Introduce a dummy source vertex s and a dummy sink vertex t , and construct the following flow network:

- Draw an edge from the source s to each vertex $\ell_i \in L$ with capacity 1.
- For every edge $(\ell_i, r_j) \in E$ in the original bipartite graph, draw an edge from ℓ_i to r_j with capacity 1.
- Draw an edge from each vertex $r_j \in R$ to the sink t with capacity 1.

On the constructed network, run any maximum-flow algorithm (e.g. Ford–Fulkerson). The resulting maximum flow from s to t gives the maximum cardinality matching of the bipartite graph.

- Step 3 of the Hungarian algorithm discussed in the lecture slides involves assigning a set of 0-cells and then determining the minimum number of lines required to cover all zeros in the current state of the (reduced cost) matrix. Can a network flow algorithm be employed to carry out Step 3 as an alternative? Justify your answer.

Ans: 6

Yes.

Construct a bipartite graph $G(L \cup R, E)$, where $L = \{\ell_1, \ell_2, \dots, \ell_N\}$ and $R = \{r_1, r_2, \dots, r_N\}$.

The left set of vertices corresponds to the rows of the matrix, and the right set of vertices corresponds to the columns. The edge set E is defined based on the 0-cells of the reduced cost matrix. Specifically, an edge $\ell_i \rightarrow r_j$ exists in this construction if and only if $c'_{ij} = c_{ij} - u_i - v_j = 0$.

This specific bipartite graph is called an *equality graph* based on which the values of dual variables are augmented in each iteration of the Hungarian algorithm, until termination condition is reached. (Note: this construct is *analogous* to the residual graph construction in the iterations of Ford–Fulkerson algorithm that formed the basis of finding augmenting flow.)

Step 3 of the Hungarian algorithm is about finding a maximum-cardinality matching in the equality graph (which correspond to the assignments of 0-cells in steps 3.1-3.2) and then a minimum vertex cover (which corresponds to the minimum number of lines covering all 0-cells in the reduced cost matrix in steps 3.3-3.4).

Both can be obtained by constructing a corresponding flow network (as described in general in the previous question) on each iteration's equality bipartite graph and then running a maximum-flow algorithm on it. The resulting maximum flow gives the maximum matching that corresponds to the $\ell_i \rightarrow r_j$ edges that get a unit (1/1) flow assignment upon termination.

The resultant min-cut $C = (C_L, C_R)$, with source $s \in C_L$ and sink $t \in C_R$, can then be used to construct the minimum vertex cover. Specifically, $L \cap C_R$ gives the set of vertices of L that are unreachable from the source s (equivalent to the horizontal lines covering unmarked rows in the Hungarian). $R \cap C_L$ gives the set of vertices of R that are reachable from s (equivalent to the vertical lines covering columns).

- (This question concerns modelling the following as a network flow problem.) The coordinator of Monash Faculty of IT's IBL program has a cohort of n students wanting to be placed with $m < n$ companies. Each company $1 \leq i \leq m$ has at most x_i available places for Monash students. Each student nominates exactly k companies (in no particular order) that they wish to be placed with.

After interviews, each company nominates $y_i > x_i$ students (again in no particular order) whom they would be happy to host, subject to their capacity constraint x_i .

Taking the respective student–company nominations into account, the IBL coordinator's goal is to assign the maximum number of students

to mutually agreeable companies. Can you help the IBL coordinator address this problem?

Ans: 7

Consider a bipartite graph $G(L \cup R, E)$ where $L = \{\ell_1, \ell_2, \dots, \ell_n\}$ represents the set of students and $R = \{r_1, r_2, \dots, r_m\}$ represents the set of companies. An edge $(\ell_i, r_j) \in E$ exists if and only if the student i nominates the company j or vice versa.

Introduce a dummy source vertex s and a dummy sink vertex t , and construct the flow network (that varies slightly in construction from how it was done in Q5):

- Draw an edge from the source s to each vertex $\ell_i \in L$ with capacity 1.
- For every edge $(\ell_i, r_j) \in E$, draw a edge from ℓ_i to r_j with capacity 1.
- Draw an edge from each vertex $r_j \in R$ to the sink t with **capacity x_j** .

A maximum flow in this network yields an assignment that maximises the number of students placed at mutually nominated companies.

```
--o0o--
  END
--o0o--
```